Module 4

- Coin Toss/Cards
- Random Number generators

Middle Square Method, Linear Congruential method, Testing the randomness

- Calculation of pi
- Transformation Methods of Special Distributions
- Random walk
- Multidimensional integration

- Coin Toss/Cards
- Random Number generators

Middle Square Method, Linear Congruential method, Testing the randomness

- Calculation of pi
- Transformation Methods of Special Distributions
- Random walk
- Multidimensional integration

- Histogram

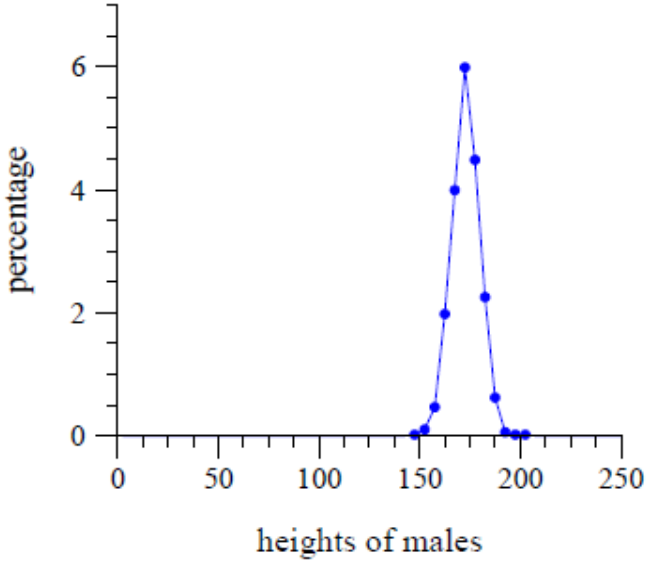
Power laws, Pareto distributions and Zipf's law

M. E. J. Newman

Department of Physics and Center for the Study of Complex Systems, University of Michigan, Ann Arbor, MI 48109, U.S.A.

- Many of the things that scientists measure have a typical size or "scale"—a typical value around which individual measurements are

- **Histogram** of people's heights



Heights in centimetres of American males. Data from the National Health Examination Survey, 1959–1962 (US Department of Health and Human Services).

https://aaronclauset.github.io/powerlaws/data.htm

Histogram : Linear binning

Key Steps in Linear Binning

1. **Define the Range**: Determine the minimum and maximum values of the dataset.
2. **Divide into Bins**: Split the range into equal-sized intervals or bins.
3. **Assign Data Points**: Place each data point into the corresponding bin based on its value.
4. **Aggregate**: Optionally, compute aggregate statistics (e.g., count, sum, or average) for each bin

Histogram : Linear binning

Key Steps in Linear Binning

1. Define the Range: Determine the minimum and maximum values of the dataset.
2. Divide into Bins: Split the range into equal-sized intervals or bins.
3. Assign Data Points: Place each data point into the corresponding bin based on its value.
4. Aggregate: Optionally, compute aggregate statistics (e.g., count, sum, or average) for each bin

Suppose you have a dataset of values:

$$[2, 5, 7.3, 13, 15]$$

- **Range:** Minimum = 2, Maximum = 15
- **Number of Bins:** 3
- **Bin Width:** (15−2)/3=4.33 (round to 5 for simplicity)

Histogram : Linear binning

Key Steps in Linear Binning

1. Define the Range: Determine the minimum and maximum values of the dataset.
2. Divide into Bins: Split the range into equal-sized intervals or bins.
3. Assign Data Points: Place each data point into the corresponding bin based on its value.
4. Aggregate: Optionally, compute aggregate statistics (e.g., count, sum, or average) for each bin

Suppose you have a dataset of values:

$$[2, 5, 7.3, 13, 15]$$

- **Range**: Minimum = 2, Maximum = 15
- **Number of Bins**: 3
- **Bin Width**: (15−2)/3=4.33 (round to 5 for simplicity)

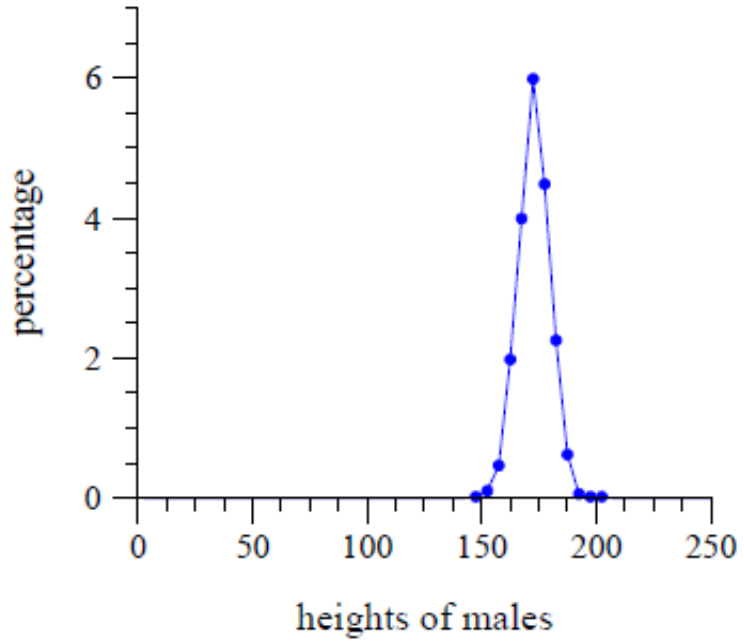
Data points in bins:
- Bin 1: [2, 5]
- Bin 2: [7.3]
- Bin 3: [13, 15]

Histogram : Linear binning

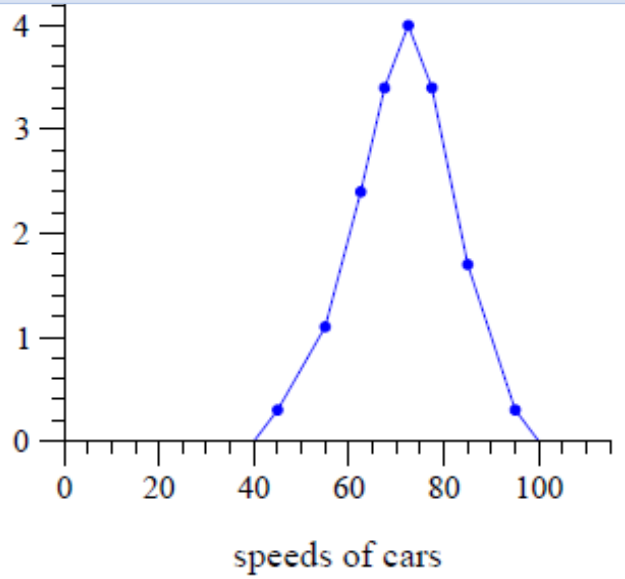
Applications

- **Histograms**: To visualize the distribution of data.
- **Data Compression**: Reducing the number of data points by grouping them into bins.
- **Signal Processing**: Discretizing continuous signals.
- **Numerical Methods**: Approximating functions or integrals.

- Many of the things that scientists measure have a typical size or “scale”—a typical value around which individual measurements are centred.



Again the histogram of speeds is strongly peaked, in this case around 75mph.



Right: histogram of speeds in miles per hour of cars on UK
motorways. Data from Transport Statistics 2003 (UK Department for Transport).

Power laws, Pareto distributions and Zipf's law

M. E. J. Newman

Department of Physics and Center for the Study of Complex Systems, University of Michigan, Ann Arbor, MI 48109, U.S.A.

But not all things we measure are peaked around a typical value. Some vary over an enormous dynamic range, sometimes many orders of magnitude!!!!!

- **frequency table** of words in a text.

$$\text{word frequency} \propto \frac{1}{\text{word rank}}.$$

Zipf's law (/zɪf/; German pronunciation: [tsɪpf]) is an empirical law stating that when a list of measured values is sorted in decreasing order, the value of the n-th entry is often approximately inversely proportional to n.

George Kingsley Zipf (Statistics, linguistics)

Monte Carlo Methods

Numerical methods which make use of random numbers are called *Monte-Carlo methods*—after the famous casino. The obvious applications of such methods are in *stochastic physics*: *e.g.*, statistical thermodynamics. However, there are other, less obvious, applications: *e.g.*, the evaluation of multi-dimensional integrals.

The modern variant of the "Monte Carlo Methods" can be traced back to the Los Alamos Laboratory in the 1940s, where it was originally developed to assist with modeling the nuclear fission process, in particular, simulating the mean free path of neutrons. Rather than solve the diffusion path of neutrons deterministically, a statistical sampling approach was applied — and it worked brilliantly.

Monte Carlo Methods

Monte Carlo methods, or **Monte Carlo experiments**, are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. The underlying concept is to use randomness to solve problems that might be deterministic in principle. They are often used in physical and mathematical problems and are most useful when it is difficult or impossible to use other approaches. In physics-related problems, Monte Carlo methods are useful for simulating systems with many coupled degrees of freedom, such as fluids, disordered materials, strongly coupled solids, and cellular structures (cellular Potts model, interacting particle systems, McKean–Vlasov processes, kinetic models of gases, rock-paper-scissor dynamics).

Other examples include modeling phenomena with significant uncertainty in inputs such as the calculation of risk in business and, in mathematics, evaluation of multidimensional definite integrals with complicated boundary conditions.

In principle, Monte Carlo methods can be used to solve any problem having a probabilistic interpretation. By the law of large numbers, integrals described by the expected value of some random variable can be approximated by taking the empirical mean (a.k.a. the sample mean) of independent samples of the variable.

When the probability distribution of the variable is parametrized, mathematicians often use a Markov chain Monte Carlo (MCMC) sampler. The central idea is to design a judicious Markov chain model with a prescribed stationary probability distribution. That is, in the limit, the samples being generated by the MCMC method will be samples from the desired (target) distribution.By the ergodic theorem, the stationary distribution is approximated by the empirical measures of the random

Monte Carlo Methods

Every game in a casino is a game of chance relying on random events such as ball falling into a particular slot on a roulette wheel, being dealt useful cards from a randomly shuffled deck, or the dice falling the right way!

In principle, any method that uses random numbers to examine some problem is a Monte Carlo method.

Basic Ingredients of Monte Carlo Algorithm

- *Random number generator*
- *Sampling rule*
- *Probability Distribution Functions (PDF)*
- *Error estimation*

History of Random Number Generators (RNGS)

Dice

Forerunne

Earliest six

Found in b

Marked w

systems

Playing Cards

Used in Cl

century

Coins

Introduced

Popular in

~1900, Er

resulting in

Spinning Wh

Ancient Greeks spun a shield balanced on the point of a spear. The shield was marked into section, and players would bet on where the shield would stop.

Evolved into fairground wheel of fortune, then into roulette

Last century evaluation of (RNGS)

“True” random number generators

- Unpredictable (in theory), but slow and possibly biased
- Usually based on physical processes, which may be microscopic or macroscopic

Examples:

  - Cards, coins, dice, roulette wheels
  - Radioactive decay, thermal noise, photoelectric effect
  - Keyboard stroke timing, lava lamps, fishtank bubbles

- Macroscopic processes are subject to Newtonian laws of motion and are therefore deterministic; they appear unpredictable, however, because we cannot determine the initial conditions with sufficient accuracy
- RNG’s based on quantum properties are completely unpredictable

Last century evaluation of (RNGS)

“Pseudo” random number generators

Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin. For, as has been pointed out several times, there is no such thing as a random number — there are only methods to produce random numbers, and a strict arithmetic procedure of course is not such a method.

... probably ... can not be justified, but should merely be judged by their results. Some statistical study of the digits generated by a given recipe should be made, but exhaustive tests are impractical. If the digits work well on one problem, they seem usually to be successful with others of the same type.

John von Neumann, 1951

Last century evaluation of (RNGS)

“Pseudo” random number generators

if we know r_m and its preceding elements, it is always possible to figure out r_{m+1}. For this reason, computers are said to generate pseudorandom numbers (yet with our incurable laziness we won’t bother saying “pseudo” all the time). While more sophisticated generators do a better job at hiding the correlations, experience shows that if you look hard enough or use these numbers long enough, you will notice correlations.

John von Neumann

- Born 1903 in Budapest
- With Stanislaw Ulam, developed a formal foundation f
  Monte Carlo method
- In 1951, proposed one of the first pseudo-RNG algorithms for use on an electronic computer:

Middle-Square Method

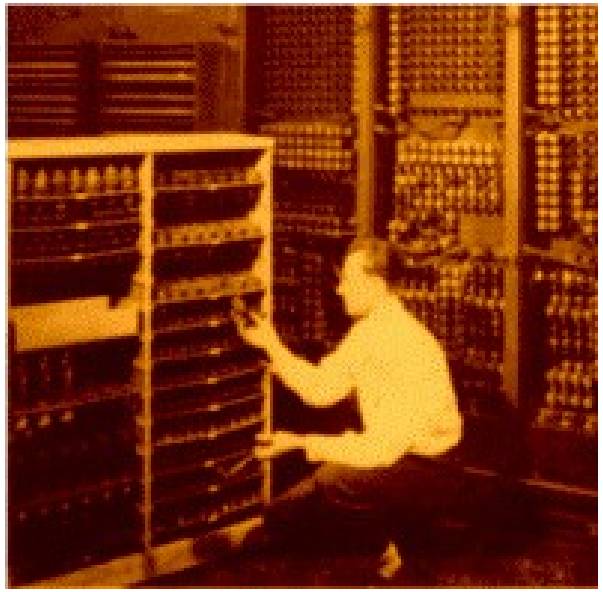
- Start with x_0, n digits long
- X_1 = middle n or n+1 digits of x_0^2

Example:

$x_0 = 157$

$157^2 = 24649$

$x_1 = 464$



ENIAC, completed in 1945

John vor

- Born 1903 ir
- With Stanisl
  Monte Carlc
- In 1951, pro
  use on an el

on fo

orithms for

I

i	Z_i	U_i	$Z_i \times Z_i$
0	7182	-	51581124
1	5811	0,5811	33767721
2	7677	0,7677	58936329
3	9363	0,9363	87665769
4	6657	0,6657	44315649
5	3156	0,3156	09960336
6	9603	0,9603	92217609
7	2176	0,2176	04734976
8	7349	0,7349	54007801
9	78	0,0078	00006084
10	60	0,006	00003600
11	36	0,0036	00001296
12	12	0,0012	00000144
13	1	0,0001	00000001
14	0	0	00000000
15	0	0	00000000

- Start with x_0, n
- X_1 = middle n c

Example:

x_0 = 15

157^2 =

x_1 = 46

ENIAC, completed in 1945

Modern “Pseudo” random number generators

- **Linear congruential generators**
  - Form: $\mathbf{x_n = a * x_{n-1} + b \pmod{M}}$
  - Most common type of PRNG in use today
  - Period depends on M, but can be as long as 10^9 when using 32-bit words
  - Disadvantage: serial correlation, which means successive numbers are not equidistributed (equally likely to fall anywhere in the range)
  - Not suitable for Monte Carlo simulations

Modern "Pseudo" random number generators

- **Linear congruential generators**
  - Form: $x_n = a * x_{n-1} + b \pmod{M}$
  - Most common type of PRNG in use today
  - Period depends on M, but can be as long as 10^9 when using 32-bit words
  - Disadvantage: serial correlation, which means successive numbers are not equidistributed (equally likely to fall anywhere in the range)
  - Not suitable for Monte Carlo simulations
- **Mersenne Twister**
  - Developed in 1997 by Makoto Matsumoto and Takuji Nishimura
  - Very fast, Negligible serial correlation
  - Period is $2^{19937} - 1$ (a Mersenne prime), Suitable for Monte Carlo, but not cryptography
- **Blum Blum Shub**
  - proposed in 1986 by Lenore Blum, Manuel Blum, and Michael Shub
  - Not very fast, so poor choice for simulations
  - Good for cryptography because of the difficulty of finding any non-random patterns through calculation (strong security proof)

Monte Carlo Methods : Random Number Generators

Random Numbers

No numerical algorithm can generate a truly random sequence of numbers.

But we can generate pseudo random numbers *with a fairly good approximation.*

Algorithm (Linear *Congruential* Method: LCM)

Aim: M number of integer sequences which are fairly (randomly) distributed in the range 0 to $M-1$.

Monte Carlo Methods : Random Number Generators

Algorithm (Linear *Congruential* Method) modulo arithmetic

Aim: M number of integer sequences which are fairly (randomly) distributed in the range 0 to $M-1$.

Formula: An iterative map like formula

$$I_{j+1} = (A\, I_j + C) \bmod M,$$

Where A, C, and M are positive integers.

- I_j is the sequence of random number
- A is the multiplier
- C is the increment
- mod is the modulo
- I_0 is the seed

Monte Carlo Methods : Random Number Generators

Algorithm (Linear *Congruential* Method) modulo arithmetic

Aim: M number of integer sequences which are fairly (randomly) distributed in the range 0 to $M-1$.

Formula: An iterative map like formula

$$I_{j+1} = (A\, I_j + C) \bmod M,$$

Where A, C, and M are positive integers.

- I_j is the sequence of random number
- A is the multiplier
- C is the increment
- mod is the modulo
- I_0 is the seed

For the LCM to have a **full period** (i.e., generate all possible values before repeating), these conditions must be met:

•C and M must be **relatively prime** (gcd = 1).

•A−1 must be **divisible by all prime factors** of M.

•**Choose** M as a **large prime number** or a **power of 2**

Monte Carlo Methods : Random Number Generators

Algorithm (Linear *Congruential* Method) modulo arithmetic

Aim: M number of integer sequences which are fairly (randomly) distributed in the range 0 to $M-1$.

Formula: An iterative map like formula

$$I_{j+1} = (A\, I_j + C) \bmod M,$$

Seed: $I(0)$

Example: $A = 7, C = 0,$ and $M = 20$ (Show first 10 outputs)

```
A=7; C=0; M=20; %%% Bad choices
rng_c=[]; I(1)=1;
for j=1:10;
I(j+1)=mod((A*I(j)+C),M)
rng_c=[rng_c, I];
end
```

Output: 1 7 9 3 1 7 9 3 1 7 9

Monte Carlo Methods : Random Number Generator

Algorithm (Linear *Congruential* Method) modulo arithmetic

Aim: M number of integer sequences which are fairly (randomly) distributed in the range 0 to $M-1$.

Formula: An iterative map like formula

$$I_{j+1} = (A\, I_j + C) \bmod M,$$

Seed: $I(0)$

```
clear all;
%%%another choice %%%
A=106; C=1283; M=6075; I(1)=1; rng_c=[];
iter=2000;
I=zeros(1,iter);
for j=1:iter;
I(j+1)=mod((A*I(j)+C),M);
rng_c=[I];
end
subplot(2,2,1);plot(rng_c(1:end1),rng_c(2:end),'.','Marker','.','MarkerSize',4)
xlabel('$I_i$');
ylabel('$I_{i+1}$');
subplot (2,2,2);hist(rng_c);
title ('Histogram');
%%figure;
subplot(2,2,[3 4]);plot(rng_c(1:end-1))
xlabel('$i$');
ylabel('$I_{i}$');
```

Monte Ca ... ator

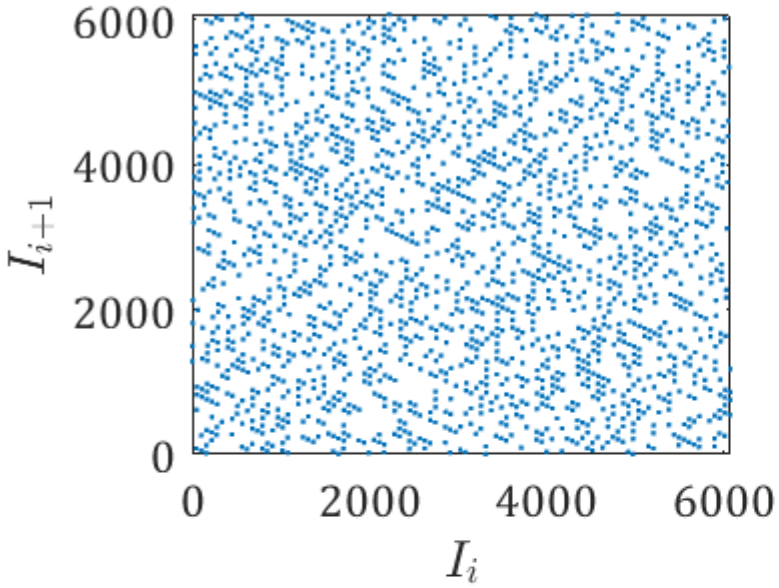
Algorithm (I ...

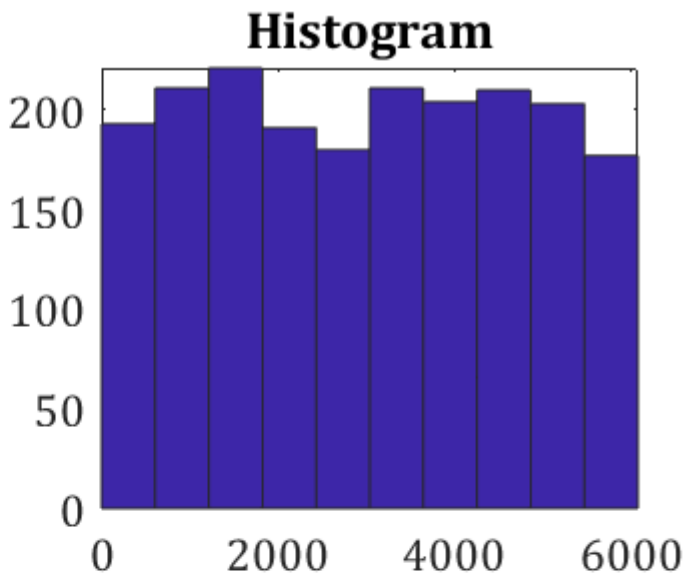
Aim: $\boldsymbol{M}$ numb ... ted in the range 0 to ...

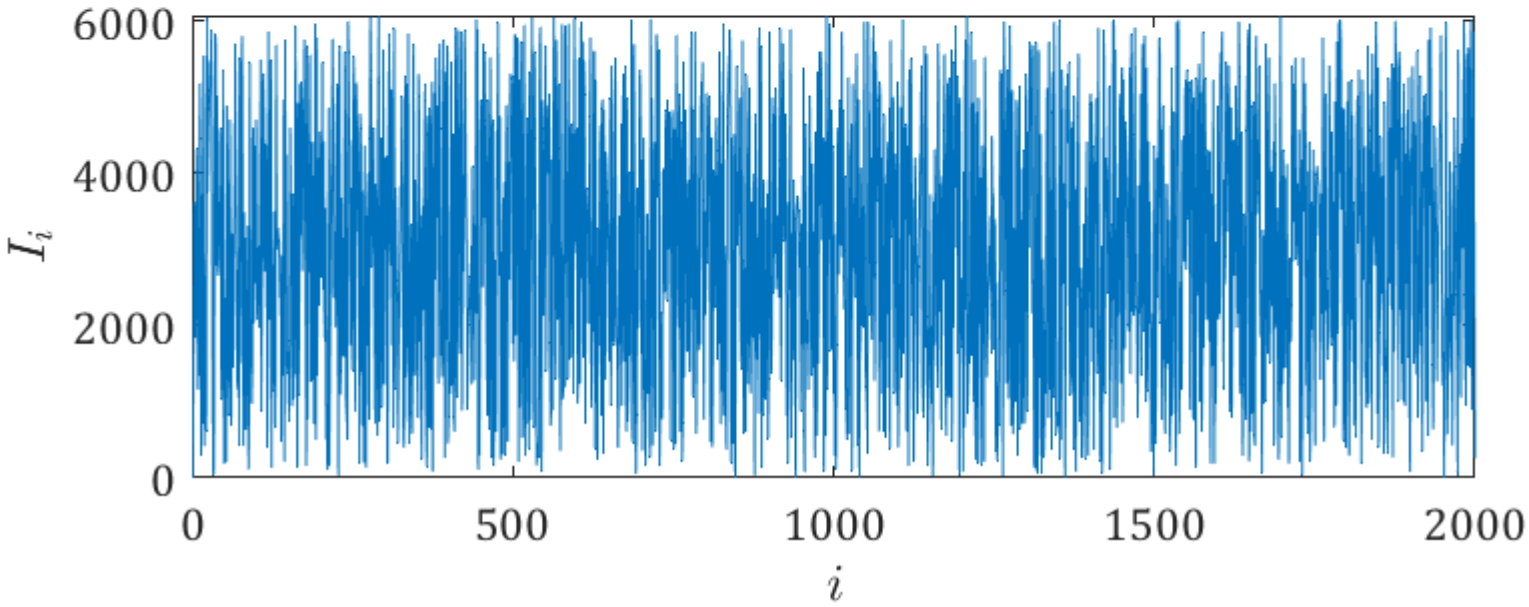
Formula: An it ...

$$\boldsymbol{I_{j+1}} = (\boldsymbol{A}\,\boldsymbol{I_j} +$$

Seed: $\boldsymbol{I(0)}$







ker','.','MarkerSize',4)

Monte Carlo Methods : Random Number Generator

Statistical Testing

- Average value: If all are rescaled at 0 to 1 i.e $S_i \in [0,1]$

Then $\bar{S} = Lt_{N\to\infty} \frac{1}{N} \sum_{i=1} S_i = \frac{1}{2}$

Monte Carlo Methods : Random Number Generator

Statistical Testing

- Average value: If all are rescaled at 0 to 1 i.e $S_i \in [0,1]$

Then $\bar{S} = Lt_{N\to\infty} \frac{1}{N} \sum_{i=1} S_i = \frac{1}{2}$

- You can also calculate the higher moment

$$\langle x^k \rangle = \frac{1}{N} \sum_{i=1}^{N} x_i^k .$$

If the numbers are distributed uniformly, then the above expression is approximately the moment of the distribution function P(x):

$$\frac{1}{N} \sum_{i=1}^{N} x_i^k \simeq \int_0^1 dx\, x^k P(x) \simeq \frac{1}{k+1} + O\left(\frac{1}{\sqrt{N}}\right).$$

Monte Carlo Methods : Random Number Generator

We wish to construct random variable with uniform distribution within the range x_1 and x_2. Then

$x = x_1 + (x_2 - x_2) * \text{RAND}$

Here RAND is between 0 to 1

There are two basic methods of constructing non-uniformly distributed random variables: *i.e.*, the *transformation method* and the *rejection method*.

Monte Carlo Method, Application: Pi calculation

Pseudo code

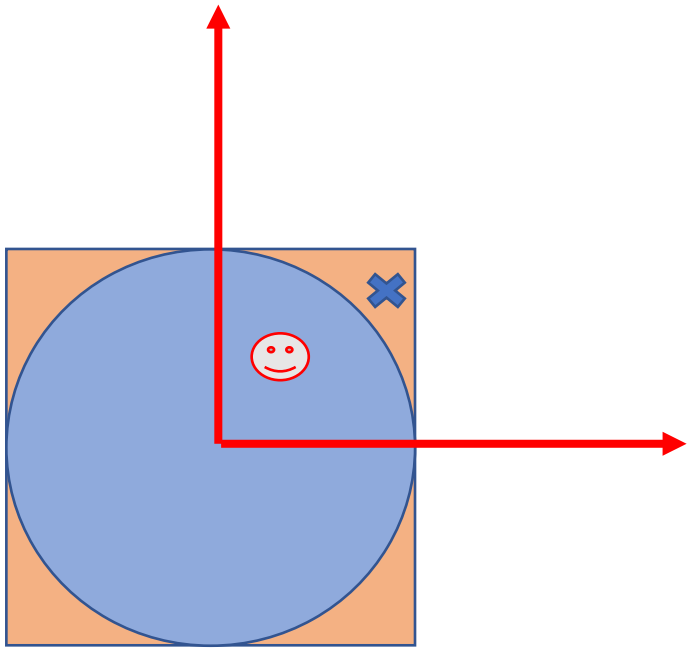
```
FUNCTION estimate_pi(n)
   count_inside_circle ← 0

   FOR i FROM 1 TO n DO
      x ← RANDOM(0, 1)   // Generate random x in [0,1]
      y ← RANDOM(0, 1)   // Generate random y in [0,1]

      IF (x^2 + y^2 ≤ 1) THEN
         count_inside_circle ← count_inside_circle + 1
      ENDIF
   ENDFOR

   pi_approx ← 4 × (count_inside_circle / n)
   RETURN pi_approx
ENDFUNCTION

// Run the function with a large number of points
PRINT estimate_pi(1,000,000)
```



Using quarter circle

- If (0,0) is the center and 1 is the radius,
- Area of the square : $1 \times 1 = 1$
- Area of the circle: $= \frac{\pi}{4}$
- Ratio: $\frac{\pi}{4}$. Thus, $\frac{\pi}{4}$= num of smile/trials;

Monte Carlo Method, Application: Pi calculation

Pseudo code

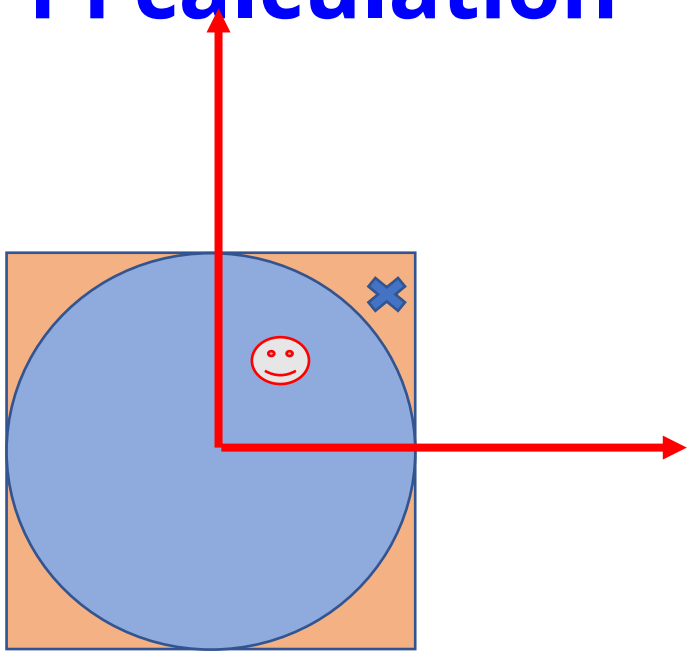
```
FUNCTION estimate_pi(n)
   count_inside_circle ← 0

   FOR i FROM 1 TO n DO
      x ← RANDOM(0, 1)   // Generate random x in [0,1]
      y ← RANDOM(0, 1)   // Generate random y in [0,1]

      IF (x^2 + y^2 ≤ 1) THEN
         count_inside_circle ← count_inside_circle + 1
      ENDIF
   ENDFOR

   pi_approx ← 4 × (count_inside_circle / n)
   RETURN pi_approx
ENDFUNCTION

// Run the function with a large number of points
PRINT estimate_pi(1,000,000)
```



- **Initialize** a counter for points inside the quarter-circle.
- **Loop** through n random points, generating (x, y) coordinates.
- **Check** if the point lies inside the circle using $x^2 + y^2 \leq 1$
- **Estimate π** using the formula:

$$\pi \approx \frac{\text{points inside the circle}}{\text{points total generated}}$$

Calculation of $\sqrt{n}$

- Sample size M
- Generate random numbers x in a range $[0, n]$
- Calculate x^2 .

$$\sqrt{n} \approx n \times \frac{\text{Count where } x^2 \leq n}{\text{Total counts } (M)}$$

Calculation of $\sqrt{n}$

- Sample size M
- Generate random numbers x in a range $[0, n]$
- Calculate x^2 .

$$\sqrt{n} \approx n \times \frac{\text{Count where } x^2 \leq n}{\text{Total counts } (M)}$$

Input: n (number), M (number of random samples)
Output: Estimated sqrt(n)

1. Initialize count = 0
2. For i from 1 to M:
   a. Generate a random number X between 0 and n
   b. If X^2 ≤ n:
      Increment count
3. Compute estimated_sqrt = (count / M) * n
4. Return estimated_sqrt

Calculation of $\sqrt{n}$

- Sample size M
- Generate random numbers x in a range $[0, n]$
- Calculate x^2 .

$$\sqrt{n} \approx n \times \frac{\text{Count where } x^2 \leq n}{\text{Total counts } (M)}$$

Input: n (number), M (number of random samples)
Output: Estimated sqrt(n)

1. Initialize count = 0
2. For i from 1 to M:
   a. Generate a random number X between 0 and n
   b. If X^2 ≤ n:
      Increment count
3. Compute estimated_sqrt = (count / M) * n
4. Return estimated_sqrt

Why Does This Work?

- If we assume that the density of squared values follows a uniform distribution, then the probability of $X_i^2 \leq n$ gives a way to extract $\sqrt{n}$.
- The expected value of numbers satisfying this condition should be close to $\sqrt{n}$.

Calculation of $\sqrt{n}$

```
clear, clc, close all

% initialization
N = 1e6;
n=36;
% generate a set of random numbers x(0, n)
x = n*rand(1, N);

% find all x^2 < n
z = (x.^2 < n);
sqrt(n) %crosscheck
sqrt2est = n*nnz(z)/N; %% Number of non-zero
elements
err = (sqrt2est - sqrt(n))/sqrt(n)*100;
```

What about $n^{\frac{1}{p}}$?

What about $n^{\frac{1}{p}}$?

Input: n (number), p (root power), M (number of random samples)
Output: Estimated n^(1/p)

1. Initialize count = 0
2. For i from 1 to M:
    a. Generate a random number X between 0 and n
    b. If X^p ≤ n:
        Increment count
3. Compute estimated_root = (count / M) * n
4. Return estimated_root

Transformation Methods of Special Distributions

- For a certain class of distributions it is possible to create pseudo-random numbers from uniformly distributed random numbers by finding a mathematical transformation.
- It works particularly nicely for the most common distributions (exponential, Poisson and normal distribution).
- It is not always feasible - this depends on the analytical description of the distribution.

Transformation Methods of Special Distributions

- For a certain class of distributions it is possible to create pseudo-random numbers from uniformly distributed random numbers by finding a mathematical transformation.
- It works particularly nicely for the most common distributions (exponential, Poisson and normal distribution).
- It is not always feasible - this depends on the analytical description of the distribution.

The idea is to find the <u>equivalence</u> between area slices of the uniform distribution P_U and the distribution of interest.

Let

$$X \sim \text{Uniform}(a, b)$$

The **PDF** is:

$$f(x) = \begin{cases} \dfrac{1}{b-a}, & a \leq x \leq b \\ 0, & \text{otherwise} \end{cases}$$

By definition, the **CDF** is:

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f\,(t)\,dt$$

Transformation Methods of Special Distributions

- For a certain class of distributions it is possible to create pseudo-random numbers from uniformly distributed random numbers by finding a mathematical transformation.
- It works particularly nicely for the most common distributions (exponential, Poisson and normal distribution).
- It is not always feasible - this depends on the analytical description of the distribution.

The idea is to find the <u>equivalence</u> between area slices of the uniform distribution P_U and the distribution of interest.

$U \sim \text{Uniform}(0,1)$

PDF: flat line at height 1

The uniform distribution is written as $P_U(u) = 1 \;\; \text{for } u \in [0,1]$

The height is **1** so that the *total area* is 1

CDF: where the "area" comes from

The cumulative distribution function is

$$F_U(u) = P(U \leq u)$$

CDF: $F_U(u) = \int_0^u 1 \; dt = u$

Transformation Methods of Special Distributions

$U \sim \text{Uniform}(0,1)$

PDF: flat line at height 1
The uniform distribution is written as $P_U(u) = 1 \text{ for } u \in [0,1]$

CDF: $F_U(u) = \int_0^u 1 \; dt = u$

- The area under the curve from 0 to u is exactly u
- So if you draw a random U, you are really choosing a **random area between 0 and 1**

Uniform means *equal probability for equal areas*.

Transformation Methods of Special Distributions

$U \sim \text{Uniform}(0,1)$

PDF: flat line at height 1
The uniform distribution is written as $P_U(u) = 1 \text{ for } u \in [0,1]$

CDF: $F_U(u) = \int_0^u 1 \; dt = u$

- The area under the curve from 0 to u is exactly u
- So if you draw a random U, you are really choosing a **random area between 0 and 1**

Uniform means *equal probability for equal areas*.

Fix a random area
Now freeze one realization:

$$U = u$$

This number u is not a value of the target variable yet - it's just an area.

Transformation Methods of Special Distributions

$U \sim \text{Uniform}(0,1)$

PDF: flat line at height 1

The uniform distribution is written as $P_U(u) = 1 \text{ for } u \in [0,1]$

CDF: $F_U(u) = \int_0^u 1 \; dt = u$

Fix a random area

Now freeze one realization:

$$U = u$$

Let the target variable be X with CDF:

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt$$

Key properties:

- $F_X(x)$ is increasing
- It maps values $x \rightarrow$ areas in $[0, 1]$

So now ask the *inverse question*:

For what value of x does the target distribution accumulate the same area u?

Mathematically:

$$F_X(x) = u$$

Transformation Methods of Special Distributions

Let the target variable be X with CDF:

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt$$

Key properties:

- $F_X(x)$ is increasing
- It maps values $x \rightarrow$ areas in $[0, 1]$

So now ask the *inverse question*:

For what value of x does the target distribution accumulate the same area u?

Mathematically:

$$F_X(x) = u$$

Inverse transform sampling finds the value in the target distribution that encloses the same probability area as a uniform draw.

Invert the C

Solve:

$$x = F_X^{-1}(u)$$

This means:

- Same probability mass
- Different geometry
- Different spacing of values

Areas are preserved, **values are reshaped**.

Transformation Methods of Special Distributions

InInverse Transform Sampling (ITS) allows us to generate samples from any continuous probability distribution using a uniform random variable. The core idea revolves around integration and the properties of the cumulative distribution function (CDF).

Step 1: Understanding the CDF

Let X be a random variable with a probability density function (PDF) $f(x)$. The cumulative distribution function (CDF) is given by:

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f(t)\,dt$$

where:

- $F(x)$ represents the probability that X takes a value less than or equal to x.
- $f(x)$ is the derivative of $F(x)$:

$$f(x) = \frac{d}{dx}F(x)$$

Transformation Methods of Special Distributions

Step 2: The Key Idea – Mapping a Uniform(0,1) Random Variable to Any Distribution

Suppose $U \sim \text{Uniform}(0, 1)$, meaning U is a random variable with values in $[0, 1]$ and constant density:

$$P(U \leq u) = u, \quad \text{for } u \in [0, 1]$$

We want to transform U into a new random variable X that follows a desired distribution with CDF $F(x)$.

To do this, we set the uniform variable equal to the CDF:

$$U = F(X)$$

Transformation Methods of Special Distributions

Step 2: The Key Idea – Mapping a Uniform(0,1) Random Variable to Any Distribution

Suppose $U \sim \text{Uniform}(0, 1)$, meaning U is a random variable with values in $[0, 1]$ and constant density:

$$P(U \leq u) = u, \quad \text{for } u \in [0, 1]$$

We want to transform U into a new random variable X that follows a desired distribution with CDF $F(x)$.

To do this, we set the uniform variable equal to the CDF:

$$U = F(X)$$

Since U is uniform over $[0, 1]$, solving for X gives:

$$X = F^{-1}(U)$$

This means that if we take a uniform random variable U and apply the inverse of the CDF, we get a value X that follows the target distribution.

Transformation Methods of Special Distributions

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt \qquad F_X(x) = u \qquad x = F_X^{-1}(u)$$

The Distribution

$$P(y) = ke^{-yk}.$$

Transformation Methods of Special Distributions

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt \qquad F_X(x) = u \qquad x = F_X^{-1}(u)$$

The Distribution $\quad f_Y(y) = k\, e^{-yk}$

Then $u = \int_o^y k\, e^{-y'k}\, dy'$

Transformation Methods of Special Distributions

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt \qquad F_X(x) = u \qquad x = F_X^{-1}(u)$$

The Distribution $f_Y(y) = k\, e^{-yk}$ Normalized?

Then $u = \int_o^y k\, e^{-y'k}\, dy'$

$$u = -e^{-y'k}\Big|_0^y = 1 - e^{-yk}$$

$$y = -\frac{1}{k}\ln(1-u)$$

```
iter=200000;
rng_c=rand(1,iter);
%%% Distribution %%
%y_poi=-(1/k)*(log(1-rng_c(2,:)));
y_poi=-(1/k)*(log(1-rng_c(1,:)));
 [a b]=hist(y_poi);
plot(b,a/iter,'-o'); hold on;
plot(b,k*exp(-b*k));
set(gca,'YScale','log');
```

	name	distribution $p(x) = Cf(x)$ $f(x)$	C
continuous	power law	$x^{-\alpha}$	$(\alpha-1)x_{\min}^{\alpha-1}$
	power law with cutoff	$x^{-\alpha}e^{-\lambda x}$	$\frac{\lambda^{\alpha-1}}{\Gamma(1-\alpha,\lambda x_{\min})}$
	exponential	$e^{-\lambda x}$	$\lambda e^{\lambda x_{\min}}$
	stretched exponential	$x^{\beta-1}e^{-\lambda x^{\beta}}$	$\beta\lambda e^{\lambda x_{\min}^{\beta}}$
	log-normal	$\frac{1}{x}\exp\left[-\frac{(\ln x-\mu)^2}{2\sigma^2}\right]$	$\sqrt{\frac{2}{\pi\sigma^2}}\left[\operatorname{erfc}\left(\frac{\ln x_{\min}-\mu}{\sqrt{2}\sigma}\right)\right]^{-1}$

name	random numbers
power law	$x = x_{\min}(1-r)^{-1/(\alpha-1)}$
exponential	$x = x_{\min} - \frac{1}{\lambda}\ln(1-r)$
stretched exponential	$x = \left[x_{\min}^{\beta} - \frac{1}{\lambda}\ln(1-r)\right]^{1/\beta}$
log-normal	$x_1=\exp(\rho\sin\theta),\ x_2=\exp(\rho\cos\theta)$ $\rho=\sqrt{-2\sigma^2\ln(1-r_1)},\ \theta=2\pi r_2$

Numerical Integration

- In general, a numerical integration is the approximation of a definite integration by a “weighted” sum of function values at discretized points within the interval of integration.

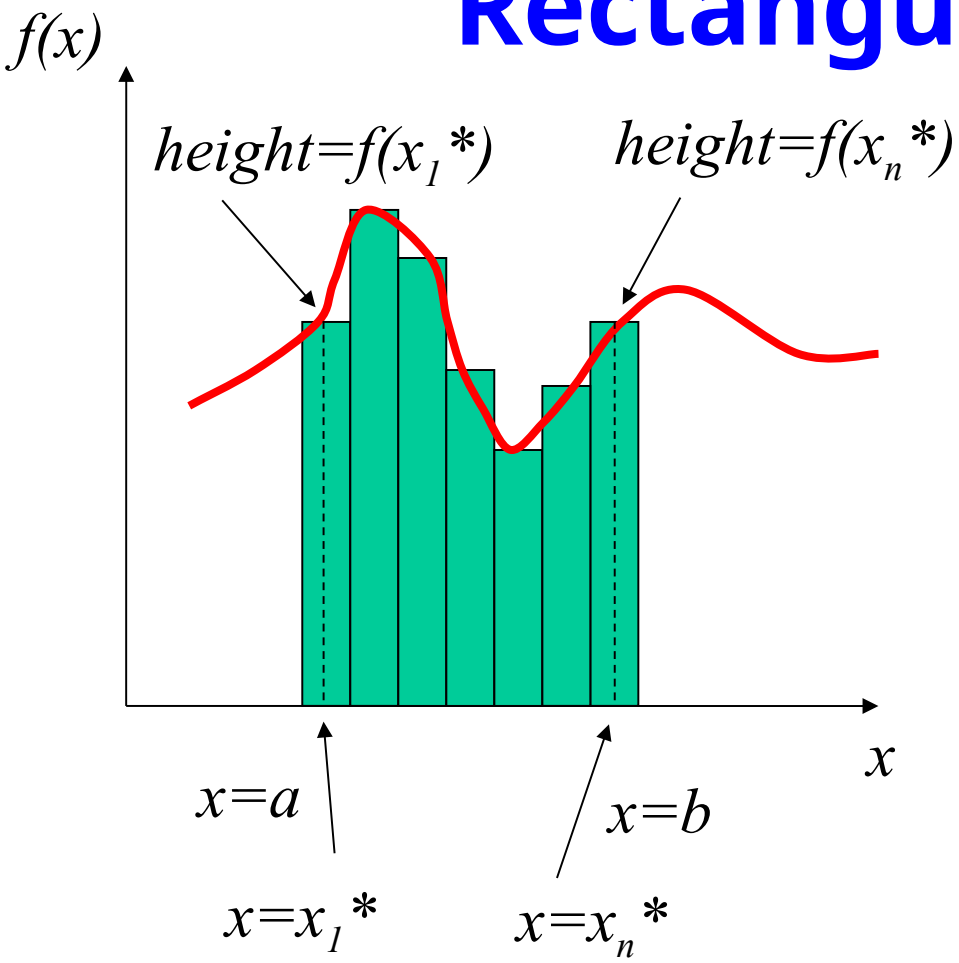
$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{N} w_i f(x_i)$$

where w_i is the weight related to the quadrature integration

scheme and $f(x)$ is the function evaluated at the

grid point x_i

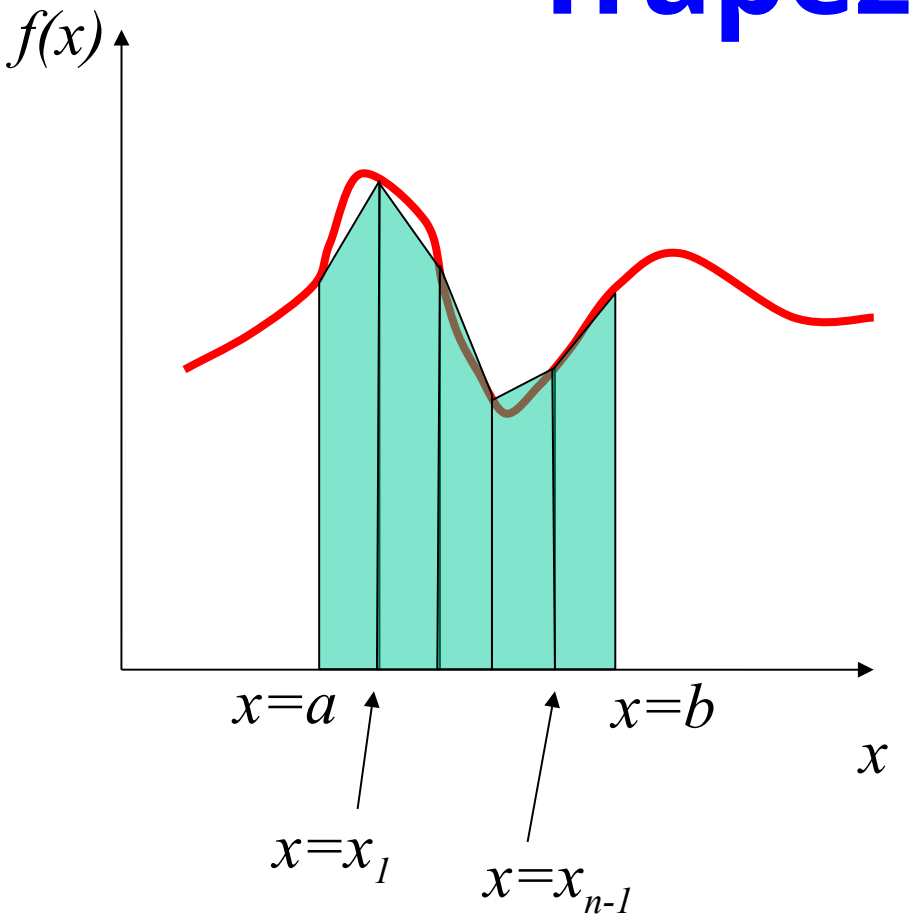
Rectangular Rule



Approximate the integration, $\int_a^b f(x)dx$, that is the area under the curve by a series of rectangles as shown. The base of each of these rectangles is $\Delta x = \frac{b-a}{n}$ and its height can be expressed as $f(x_i^*)$ where x_i^* is the midpoint of each rectangle

$$\int_a^b f(x)dx = f(x_1*)\Delta x + f(x_2*)\Delta x + .. f(x_n*)\Delta x$$
$$= \Delta x[f(x_1*) + f(x_2*) + .. f(x_n*)]$$

Trapezoidal Rule



The rectangular rule can be made more accurate by using trapezoids to replace the rectangles as shown. A linear approximation of the function locally sometimes work much better than using the averaged value like the rectangular rule does.

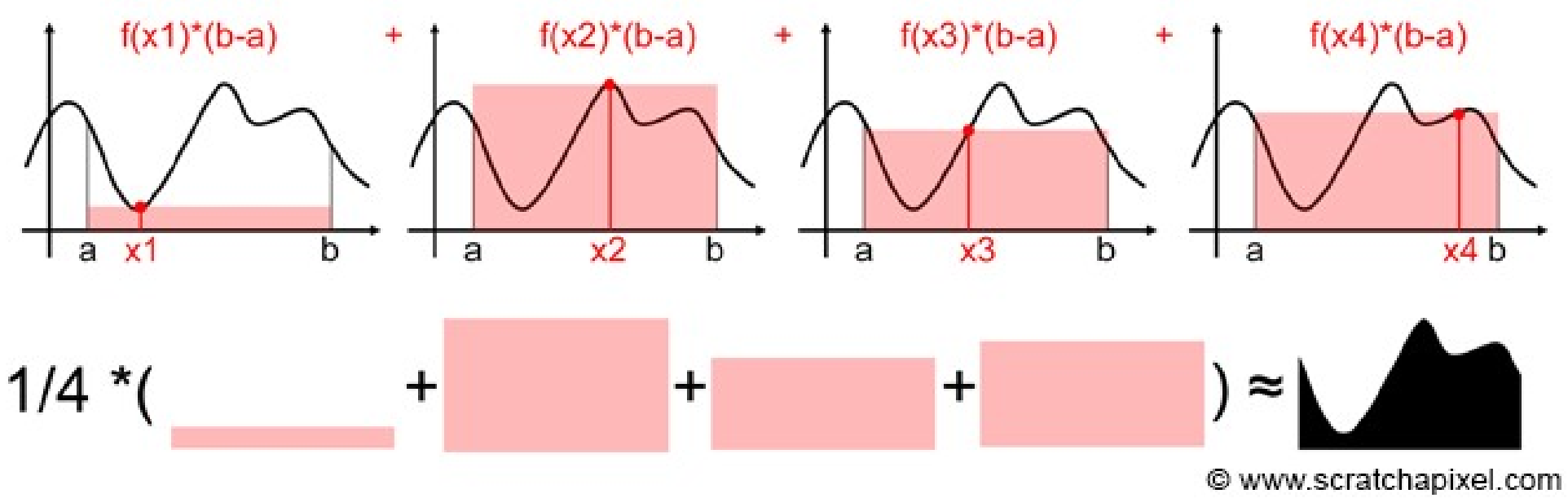
$$\int_a^b f(x)dx = \frac{\Delta x}{2}[f(a)+f(x_1)] + \frac{\Delta x}{2}[f(x_1)+f(x_2)] + .. + \frac{\Delta x}{2}[f(x_{n-1})+f(b)]$$

$$= \Delta x[\frac{1}{2}f(a) + f(x_1) + .. f(x_{n-1}) + \frac{1}{2}f(b)]$$

Monte Carlo Application: Integration

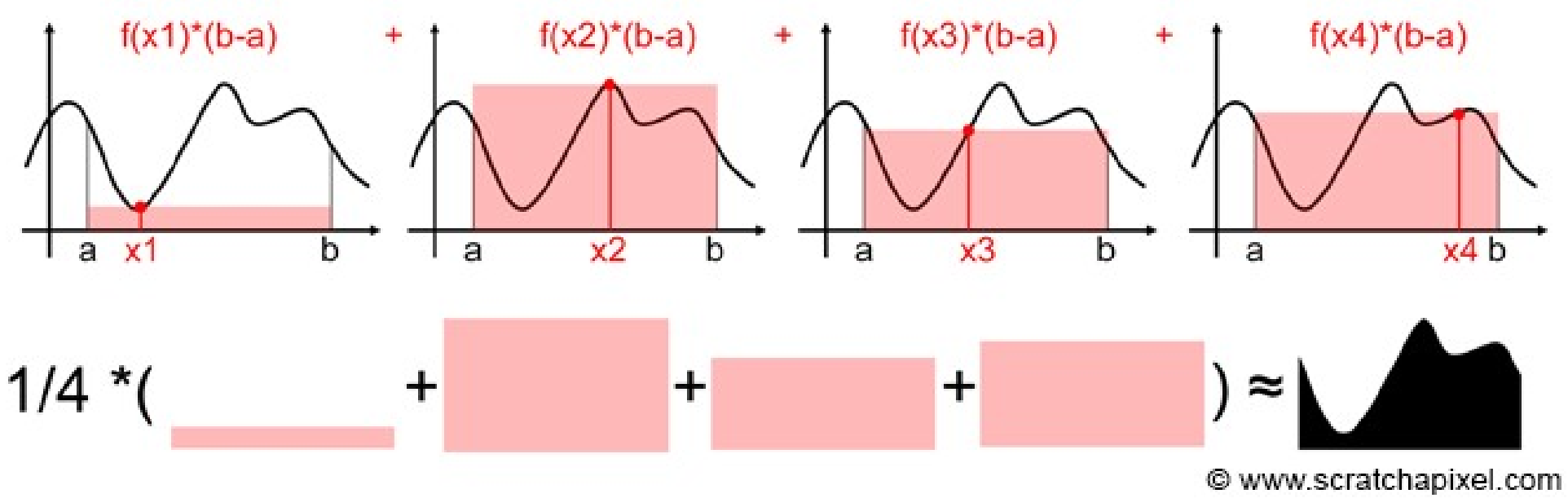
The basic Monte Carlo integration uses random sampling of a function to numerically compute an estimate of its integral. Suppose that we want to integrate the one-dimensional function $f(x)$ from a to b :

$$F = \int_a^b f(x)dx$$



https://cs.dartmouth.edu/wjarosz/publications/dissertation/appendixA.pdf

Monte Carlo Application: Integration



We keep evaluating the function at different **random points** between a and b, adding up the area of the rectangles and averaging the sum, the resulting number gets closer and closer to the actual result of the integral. It's not surprising in a way as the rectangles which are too large compensate for the rectangles which are too small.

https://cs.dartmouth.edu/wjarosz/publications/dissertation/appendixA.pdf

Monte Carlo Application: Integration

```
clear, close all
%f(x)=x;
% initialization
zz=[];
for N = [10 1000 2000 5000 10000 10^5 10^6];
a=1; b=3;
x = a+(b-a)*rand(1, N);
x1=x;
y1=x1;
area_seg=((b-a)*y1);
area=sum(area_seg)/N;
zz=[zz; N,area];
area=[];
area_seg=[];
end
Plot(zz(:,1)zz(:,2),'-o')
```

Monte Carlo Application: Integration

Given a set of N uniform random variables $X \in [a,b)$ with a corresponding PDF of $\frac{1}{b-a}$, the Monte Carlo estimator for computing F is

$$\langle F^N \rangle = (b-a)\frac{1}{N-1}\sum_{i=0} f(X_i)$$

Standard integration techniques exist which converge much faster in one dimension;
however, these techniques suffer from the *curse of dimensionality*, where the convergence rate becomes *exponentially* worse with increased dimensions. The basic Monte Carlo estimator above can easily be extended to multiple dimensions, and, in contrast to deterministic quadrature techniques, the convergence rate for Monte Carlo is independent of the number of dimensions in the integral. This makes Monte Carlo integration the only practical technique for many high dimensional integration

Monte Carlo Application: Integration

Given a set of N uniform random variables $X \in [a,\mathrm{b})$ with a corresponding PDF of $\frac{1}{b-a}$, the Monte Carlo estimator for computing F is $\langle F^N \rangle = (b-a)\frac{1}{N-1}\sum_{i=0} f(X_i)$

Expected Value

$$
\begin{aligned}
E[\langle F^N \rangle] &= E\left[(b-a)\frac{1}{N}\sum_{i=0}^{N-1} f(X_i)\right], \\
&= (b-a)\frac{1}{N}\sum_{i=0}^{N-1} E[f(X_i)], \\
&= (b-a)\frac{1}{N}\sum_{i=0}^{N-1}\int_a^b f(x)pdf(x)\,dx, \\
&= \frac{1}{N}\sum_{i=0}^{N-1}\int_a^b f(x)\,dx, \\
&= \int_a^b f(x)\,dx, \\
&= F.
\end{aligned}
$$

Random walk

The one-dimensional random walk is constructed as follows:

- You walk along a line, each pace being the same length.
- Before each step, you flip a coin.
- If it's heads, you take one step forward. If it's tails, you take one step back.
- The coin is unbiased, so the chances of heads or tails are equal.

The problem is to find the probability of landing at a given spot after a given number of steps, and, in particular, to find **how far away you are on average from where you started.**

Why do we care about this game?

The random walk is central to statistical physics. It is essential in predicting how fast one gas will diffuse into another, how fast heat will spread in a solid, how big fluctuations in pressure will be in a small container, and many other statistical phenomena.

Einstein used the random walk to find the size of atoms from the Brownian motion.

1-D random walk

"drunkard's walk"

- Each step is of length $\boldsymbol{d_0}$ **(=1)** but it can be either forward **(+1)** or backward **(-1).**
- Probability of going forward is $\boldsymbol{p}$ and backward$(\boldsymbol{q} = \mathbf{1} - \boldsymbol{p})$.

(we take $\boldsymbol{p} = \frac{\mathbf{1}}{\mathbf{2}}$)

- How far does he/she move(on average),
- what is the standard deviation?

1-D random walk

- After ***N*** steps, if n_1 are forward, the distance/net displacement is,

$$\boldsymbol{m} = [\boldsymbol{n_1} - (\boldsymbol{N} - \boldsymbol{n_1})]\boldsymbol{d_0}$$

- The probability of this occurring is,

$$\boldsymbol{P(n_1)} = \frac{\boldsymbol{N!}}{\boldsymbol{n_1!\,(N - n_1)!}}\boldsymbol{p^{n_1} q^{N-n_1}}$$

$$\boxed{\overline{\boldsymbol{n_1}} = \boldsymbol{Np}}$$

1-D random walk

The average distance covered after *N* steps is,

$$\overline{m} = (2\overline{n_1} - N)d_0$$
$$= (2pN - N)d_0$$
$$= N(2p - 1)d_0$$

$$\boxed{\overline{m} = N\overline{d}}$$

$\overline{d} = (2p - 1)d_0$ The average distance per step.

For $p = \frac{1}{2}$, $\overline{d} = 0$, and $\bar{m} = 0$

1-D random walk

$$\overline{m} = (2\overline{n_1} - N)d_0$$
$$= (2pN - N)d_0$$
$$= N(2p - 1)d_0$$

Pseudo code for mean displacement

```
INPUT:
   - N (Number of steps per walk)
   - T (Number of trials)

INITIALIZE:
   - total_displacement = 0

FOR t = 1 to T:
   - position = 0  (Starting at the origin)

   FOR n = 1 to N:
      - step = Random choice of (+1 or -1)
      - position = position + step

   - total_displacement += position  (Not absolute!)

MEAN_DISPLACEMENT = total_displacement / T

OUTPUT MEAN_DISPLACEMENT
```

1-D random walk

- The walker starts at 0
- At each step, it moves **left (-1)** or **right (+1)** with equal probability.
- The final displacement is the **absolute value** of the walker's position.
- The mean displacement is averaged over **T** trials.

🚀 **Fun fact:** The **expected mean displacement** grows roughly as √**N** due to diffusion principles!

Pseudo code for mean absolute displacement

```
INPUT:
   - N (Number of steps per walk)
   - T (Number of trials)

INITIALIZE:
   - total_displacement = 0

FOR t = 1 to T:
   - position = 0  (Starting at the origin)

   FOR n = 1 to N:
      - step = Random choice of (+1 or -1)
      - position = position + step

   - displacement = Absolute value of position
   - total_displacement += displacement

MEAN_DISPLACEMENT = total_displacement / T

OUTPUT MEAN_DISPLACEMENT
```

Pseudocode for computing the probability of landing at position x after n steps in a **1D unbiased random walk**

```
Function RandomWalkSimulatedProbability(n, x,
numTrials):
    count = 0  // Count how many times we land at x

    For trial in 1 to numTrials:
        position = 0  // Start at origin

        For step in 1 to n:
            If Random(0,1) < 0.5:
                position = position - 1  // Move left
            Else:
                position = position + 1  // Move right

        If position == x:
            count = count + 1  // Successful trials

    Return count / numTrials  // Estimated probability
```

1. Probability of landing at a given spot after n steps
2. Expected displacement from the starting point (Mean Squared Displacement, MSD)

1. Probability of Landing at a Given Spot

For a **1D unbiased random walk**, where at each step you move $+1$ or -1 with equal probability (0.5 each), the probability of being at position x after n steps is given by:

$$P_n(x) = \frac{\binom{n}{(n+x)/2}}{2^n}$$

where $\binom{n}{k}$ is the binomial coefficient. This formula works when $n + x$ is even (since the walker can only reach certain points after an even or odd number of steps).

1. Probability of landing at a given spot after n steps
2. Expected displacement from the starting point (Mean Squared Displacement, MSD)

2. Expected Distance from the Starting Point (MSD)

The mean squared displacement (MSD) in 1D follows:

$$\langle x^2 \rangle = n$$

which means the **root mean square displacement** is:

$$\sqrt{\langle x^2 \rangle} = \sqrt{n}$$

So, on average, after n steps, the walker is about $\sqrt{n}$ units away from the starting position.

1-D random walk

Standard deviation.

$$\sigma^2 = \overline{(m - \overline{m})^2} = \overline{[(2n_1 - N)d_0 - (2\overline{n_1} - N)d_0]^2}$$
$$= 4d_0^2\overline{(n_1 - \overline{n_1})^2}$$
$$= 4d_0^2 Npq$$

$$\boxed{\sigma = 2d_0\sqrt{Npq}}$$

$$\sigma^2 = N\overline{\Delta d^2} \quad \text{where} \quad 4pqd_0^2 = \overline{\Delta d^2} \quad \text{(s.d. per step)}$$

$$\boxed{p = \frac{1}{2}, q = \frac{1}{2}, pq = \frac{1}{4}, d_0 = 1\,; \sigma = \sqrt{N}}$$

https://galileo.phys.virginia.edu/classes/152.mf1i.spring02/RandomWalk.htm

https://mathworld.wolfram.com/RandomWalk1Dimensional.html

1-D random walk

$$\frac{\sigma}{\overline{m}} = \frac{2d_0\sqrt{Npq}}{N(2p-1)d_0}$$
$$= \frac{2\sqrt{pq}}{(2p-1)}\frac{1}{\sqrt{N}}$$

$$\boxed{\frac{\sigma}{\overline{m}} \propto \frac{1}{\sqrt{N}}}$$

Transformation Methods of Special Distributions

- For a certain class of distributions it is possible to create pseudo-random numbers from uniformly distributed random numbers by finding a mathematical transformation.
- works particularly nicely for the most common distributions (exponential, Poisson and normal distribution).
- It is not always feasible - this depends on the analytical description of the distribution.

Let us now consider the distribution P(y). If we compare the areas of integration, we find

$$z = \int_0^y P(y')dy' = \int_0^z P_u(z')dz'$$

where z is a uniformly distributed random variable and y a random variable distributed according to the desired distribution.

Transformation Methods of Special Distributions

$$z = \int_0^y P(y')dy' = \int_0^z P_u(z')dz'$$

Let us rewrite the integral of $P(y)$ as $I_P\,(y)$ then we find $z\ = I_P\,(y)$ and therefore

$$\boxed{y = I_P^{-1}(z)}$$

Transformation Methods of Special Distributions

$$z = \int_0^y P(y')dy' = \int_0^z P_u(z')dz'$$

Let us rewrite the integral of $P(y)$ as $I_P\,(y)$ then we find $z\ = I_P\,(y)$ and therefore

$$\boxed{y = I_P^{-1}(z)}$$

•The area under the curve from 0 to z is exactly z
•So if you draw a random U, you are really choosing a **random area between 0 and 1**
Uniform means *equal probability for equal areas*.